Aeron - Code Gen

Act as an expert Python developer. I need a Python script that functions as a small data utility to plot CSV data.
Please write a complete, readable script using pandas and matplotlib that does the following:

Instead of asking the user to type a file path, use Python's built-in tkinter.filedialog to open a native file picker so the user can visually select their CSV file.
Once the file is loaded, read the CSV and print a numbered list of all the available columns to the terminal.
Prompt the user in the terminal to select two columns (one for the X-axis, one for the Y-axis) by typing their corresponding numbers.
Prompt the user to choose the type of graph they want to generate: a Line chart or a Bar graph.
Generate and display the requested plot using matplotlib. Include basic error handling (e.g., if the user selects a column with non-numeric data for the Y-axis, print a helpful warning).
Keep the code clean, well-commented, and modular. Do not use any external libraries other than pandas and matplotlib so it is easy to run.

Here is a complete, modular Python script that meets all your requirements. It uses `tkinter` (which comes pre-installed with standard Python distributions) to handle the file selection, and uses `pandas` and `matplotlib` for the data processing and plotting.

Prerequisites

Before running the script, ensure you have the required external libraries installed. You can install them via pip if you haven't already:

Bash

```
pip install pandas matplotlib
```

The Python Script

Save the following code in a file named `csv_plotter.py` and run it from your terminal.

Python

```
import sys
import pandas as pd
import matplotlib.pyplot as plt
import tkinter as tk
from tkinter import filedialog

def select_file() -> str:
    """Opens a native file dialog for the user to select a CSV file.
    # Initialize tkinter and hide the main background window
    root = tk.Tk()
    root.withdraw()

    # Force the dialog to appear on top of other windows
    root.attributes('-topmost', True)

    print("Opening file picker...")
    file_path = filedialog.askopenfilename(
        title="Select a CSV File",
        filetypes=(("CSV files", "*.csv"), ("All files", "*.*"))
    )
    return file_path

def get_column_choice(columns: list, axis_name: str) -> str:
    """Prompts the user to select a column by its numbered index."""
    while True:
        try:
            choice = int(input(f"Select the column number for the {a
            if 1 <= choice <= len(columns):
                return columns[choice - 1]
            else:
                print(f"Invalid choice. Please enter a number betwee
        except ValueError:
            print("Invalid input. Please enter a valid number.")

def get_plot_type() -> str:
    """Prompts the user to choose between a line or bar graph."""
```

```
    while True:
        choice = input("Choose plot type ('line' or 'bar'): ").strip
        if choice in ['line', 'bar']:
            return choice
        print("Invalid choice. Please type 'line' or 'bar'.")

def plot_data(df: pd.DataFrame, x_col: str, y_col: str, plot_type: s
    """Generates and displays the requested matplotlib plot."""
    # Basic error handling: check if the Y-axis data is numeric
    if not pd.api.types.is_numeric_dtype(df[y_col]):
        print(f"\n[Warning] The selected Y-axis column '{y_col}' cor
        print("Matplotlib may fail or produce an unexpected plot. At

    # Set up the figure size
    plt.figure(figsize=(10, 6))

    try:
        # Generate the selected plot type
        if plot_type == 'line':
            plt.plot(df[x_col], df[y_col], marker='o', linestyle='-'
        elif plot_type == 'bar':
            plt.bar(df[x_col], df[y_col], color='b')

        # Add labels and formatting
        plt.title(f"{plot_type.capitalize()} Plot: {y_col} vs {x_col
        plt.xlabel(x_col)
        plt.ylabel(y_col)

        # Rotate X-axis labels in case they are long strings (like d
        plt.xticks(rotation=45, ha='right')

        # Adjust layout so labels aren't cut off
        plt.tight_layout()

        # Display the plot
        plt.show()

    except Exception as e:
        print(f"\n[Error] An error occurred while generating the plo

def main():
    print("--- CSV Data Plotting Utility ---")

    # 1. Select the file
    file_path = select_file()
    if not file_path:
        print("No file was selected. Exiting program.")
        sys.exit()

    print(f"\nLoading file: {file_path}")
```

```
# 2. Read the CSV
try:
    df = pd.read_csv(file_path)
except Exception as e:
    print(f"[Error] Failed to read the CSV file: {e}")
    sys.exit()

if df.empty:
    print("The selected CSV file is empty. Exiting program.")
    sys.exit()

# 3. Print available columns
print("\nAvailable columns:")
columns = df.columns.tolist()
for idx, col in enumerate(columns, start=1):
    print(f"  {idx}. {col}")
print("-" * 30)

# 4. Get user selections
x_col = get_column_choice(columns, "X")
y_col = get_column_choice(columns, "Y")
print("-" * 30)
plot_type = get_plot_type()

# 5. Plot the data
print("\nGenerating plot... (Close the plot window to exit the s
plot_data(df, x_col, y_col, plot_type)
```

Act as an expert Node.js backend developer. I need an Express API for a simple File Upload Portal prototype.

Please write a complete server.js file using express, multer, and cors. The API must have the following requirements:

Set up a local storage engine using multer that saves uploaded files to an /uploads directory.

Create an upload endpoint (POST /upload). It must accept a single file and three additional text fields: logDate, description, and uploaderName.

Restrict the allowed file uploads to ONLY .log, .pdf, .csv, and .zip extensions. If a user tries to upload something else, return an HTTP 400 error.

When a file is successfully uploaded, save its metadata (original file name, saved file name, logDate, description, uploaderName) into a simple local metadata.json file to act as our persistent database. If the file doesn't exist, create it.

Create an endpoint (GET /files) that reads metadata.json and returns the list of all uploaded files.

Create a download endpoint (GET /download/:filename) that allows a user to download a specific file from the /uploads folder.

Keep the code clean and well-commented. Provide the package.json dependencies I need to install.

Here is the complete Express API setup for your File Upload Portal prototype. It uses `multer` for handling the multipart/form-data requests and standard Node.js file system (`fs`) modules to manage the local storage and JSON database.

Dependencies

First, you'll need to initialize your project and install the required dependencies. You can do this by running the following commands in your terminal:

Bash

```
npm init -y
npm install express multer cors
```

The Node.js Server (`server.js`)

Create a `server.js` file and paste the following code. It handles the directory setup, file filtering, routing, and metadata management.

JavaScript

```
const express = require('express');
const multer = require('multer');
const cors = require('cors');
const fs = require('fs');
```

```
const path = require('path');

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Paths
const UPLOADS_DIR = path.join(__dirname, 'uploads');
const METADATA_FILE = path.join(__dirname, 'metadata.json');

// --- Initialization ---

// Ensure the /uploads directory exists
if (!fs.existsSync(UPLOADS_DIR)) {
    fs.mkdirSync(UPLOADS_DIR, { recursive: true });
}

// Ensure the metadata.json file exists
if (!fs.existsSync(METADATA_FILE)) {
    fs.writeFileSync(METADATA_FILE, JSON.stringify([]));
}

// Helper function to read metadata
const readMetadata = () => {
    try {
        const data = fs.readFileSync(METADATA_FILE, 'utf8');
        return JSON.parse(data);
    } catch (err) {
        console.error("Error reading metadata:", err);
        return [];
    }
};

// Helper function to write metadata
const writeMetadata = (data) => {
    try {
        fs.writeFileSync(METADATA_FILE, JSON.stringify(data, null, 2
    } catch (err) {
        console.error("Error writing metadata:", err);
    }
};

// --- Multer Configuration ---

// Configure storage engine
const storage = multer.diskStorage({
```

```
    destination: (req, file, cb) => {
        cb(null, UPLOADS_DIR);
    },
    filename: (req, file, cb) => {
        // Append a timestamp to the original filename to prevent ov
        const uniqueSuffix = Date.now() + '-' + Math.round(Math.ranc
        const ext = path.extname(file.originalname);
        const baseName = path.basename(file.originalname, ext);
        cb(null, `${baseName}-${uniqueSuffix}${ext}`);
    }
});

// Configure file filter (restrict to .log, .pdf, .csv, .zip)
const fileFilter = (req, file, cb) => {
    const allowedExtensions = ['.log', '.pdf', '.csv', '.zip'];
    const ext = path.extname(file.originalname).toLowerCase();

    if (allowedExtensions.includes(ext)) {
        cb(null, true); // Accept the file
    } else {
        cb(new Error('Invalid file type. Only .log, .pdf, .csv, and
    }
};

const upload = multer({
    storage: storage,
    fileFilter: fileFilter
});

// --- API Endpoints ---

/**
 * POST /upload
 * Accepts a single file and text fields (logDate, description, uplo
 */
app.post('/upload', (req, res) => {
    // Handle the upload manually to catch the fileFilter error clea
    const uploadHandler = upload.single('file');

    uploadHandler(req, res, (err) => {
        if (err instanceof multer.MulterError) {
            // A Multer error occurred when uploading (e.g., file to
            return res.status(400).json({ error: err.message });
        } else if (err) {
            // An unknown error occurred, or our fileFilter threw ar
            return res.status(400).json({ error: err.message });
        }

        // Check if file was actually provided
        if (!req.file) {
```

```
            return res.status(400).json({ error: 'No file uploaded.'
        }

        // Extract text fields
        const { logDate, description, uploaderName } = req.body;

        // Construct metadata object
        const newEntry = {
            id: Date.now().toString(),
            originalName: req.file.originalname,
            savedName: req.file.filename,
            size: req.file.size,
            logDate: logDate || '',
            description: description || '',
            uploaderName: uploaderName || '',
            uploadTimestamp: new Date().toISOString()
        };

        // Save to database (metadata.json)
        const metadata = readMetadata();
        metadata.push(newEntry);
        writeMetadata(metadata);

        res.status(201).json({
            message: 'File uploaded successfully!',
            file: newEntry
        });
    });
});

/**
 * GET /files
 * Returns the list of all uploaded files from metadata.json
 */
app.get('/files', (req, res) => {
    const metadata = readMetadata();
    res.json(metadata);
});

/**
 * GET /download/:filename
 * Downloads a specific file from the /uploads directory
 */
app.get('/download/:filename', (req, res) => {
    const filename = req.params.filename;
    const filePath = path.join(UPLOADS_DIR, filename);

    // Prevent directory traversal attacks
    if (!filePath.startsWith(UPLOADS_DIR)) {
        return res.status(403).json({ error: 'Access denied.' });
```

```
    }

    if (fs.existsSync(filePath)) {
        res.download(filePath, (err) => {
            if (err) {
                console.error("Error downloading file:", err);
                if (!res.headersSent) {
                    res.status(500).json({ error: 'Failed to downloa
                }
            }
        });
    } else {
        res.status(404).json({ error: 'File not found.' });
    }
});

// Start the server
app.listen(PORT, () => {
    console.log(`Server is running on http://localhost:${PORT}`);
    console.log(`Uploads directory: ${UPLOADS_DIR}`);
    console.log(`Metadata database: ${METADATA_FILE}`);
});
```

Act as an expert React developer. I need a clean, single-page React application to serve as the frontend for my File Upload Portal prototype. Please write the code for a functional component (e.g., App.jsx) that does the following:

Displays a form with a file input that explicitly only accepts .log, .pdf, .csv, .zip.
Includes text input fields for logDate (date picker), description, and uploaderName.
Handles form submission by sending a multipart/form-data POST request to http://localhost:3000/upload. Include basic error handling/alerts for success or failure.
Below the form, display a "File Dashboard" section. On component mount, it should fetch the list of files via GET http://localhost:3000/files and display them in a clean table or list.
Next to each file in the dashboard, include a "Download" button that triggers a download from http://localhost:3000/download/:filename.
Include some basic, clean CSS (either inline, standard CSS, or Tailwind classes—whichever is fastest) so it looks presentable without taking up too much time.

Keep the code modular and use React Hooks (useState, useEffect).

Here is the complete, single-page React component for your File Upload Portal.

I've used **Tailwind CSS** classes for styling, as it provides a very clean, modern UI right out of the box without needing a separate stylesheet. If you are using Vite or Create React App, just ensure Tailwind is configured in your project.

`App.jsx`

JavaScript

```
import React, { useState, useEffect } from 'react';

export default function App() {
  // --- State Hooks ---
  const [file, setFile] = useState(null);
  const [logDate, setLogDate] = useState('');
  const [description, setDescription] = useState('');
  const [uploaderName, setUploaderName] = useState('');

  const [filesList, setFilesList] = useState([]);
  const [status, setStatus] = useState({ type: '', message: '' });
  const [isSubmitting, setIsSubmitting] = useState(false);

  // --- API Base URL ---
  const API_BASE = 'http://localhost:3000';

  // --- Effects ---
  useEffect(() => {
    fetchFiles();
  }, []);

  // --- Functions ---
  const fetchFiles = async () => {
    try {
      const response = await fetch(`${API_BASE}/files`);
      if (!response.ok) throw new Error('Failed to fetch files');
      const data = await response.json();
      setFilesList(data);
    } catch (error) {
      console.error("Error fetching files:", error);
      setStatus({ type: 'error', message: 'Could not load the file c
    }
  };
```

```
  const handleFileChange = (e) => {
    if (e.target.files && e.target.files.length > 0) {
      setFile(e.target.files[0]);
    }
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!file) {
      setStatus({ type: 'error', message: 'Please select a file to u
      return;
    }

    setIsSubmitting(true);
    setStatus({ type: '', message: '' });

    // Construct FormData for multipart/form-data payload
    const formData = new FormData();
    formData.append('file', file);
    formData.append('logDate', logDate);
    formData.append('description', description);
    formData.append('uploaderName', uploaderName);

    try {
      const response = await fetch(`${API_BASE}/upload`, {
        method: 'POST',
        body: formData,
      });

      const result = await response.json();

      if (!response.ok) {
        throw new Error(result.error || 'Upload failed');
      }

      setStatus({ type: 'success', message: 'File uploaded successfu

      // Reset form fields
      setFile(null);
      setLogDate('');
      setDescription('');
      setUploaderName('');
      e.target.reset(); // Resets the file input visually

      // Refresh the dashboard
      fetchFiles();

    } catch (error) {
      console.error("Upload error:", error);
```

```
      setStatus({ type: 'error', message: error.message });
    } finally {
      setIsSubmitting(false);
    }
  };

  return (
    <div className="min-h-screen bg-gray-50 py-10 px-4 sm:px-6 lg:p
      <div className="max-w-4xl mx-auto space-y-8">

        {/* Header */}
        <div className="text-center">
          <h1 className="text-3xl font-extrabold text-gray-900">File
          <p className="mt-2 text-sm text-gray-600">Upload and manag
        </div>

        {/* Status Messages */}
        {status.message && (
          <div className={`p-4 rounded-md ${status.type === 'error'
            <p className="text-sm font-medium">{status.message}</p>
          </div>
        )}

        {/* Upload Form */}
        <div className="bg-white py-8 px-6 shadow rounded-lg sm:px-1
          <form onSubmit={handleSubmit} className="space-y-6">

            <div>
              <label className="block text-sm font-medium text-gray-
              <div className="mt-1">
                <input
                  type="file"
                  accept=".log,.pdf,.csv,.zip"
                  onChange={handleFileChange}
                  className="block w-full text-sm text-gray-500 file
                />
              </div>
            </div>

            <div className="grid grid-cols-1 gap-y-6 gap-x-4 sm:grid
              <div>
                <label className="block text-sm font-medium text-gra
                <input
                  type="date"
                  value={logDate}
                  onChange={(e) => setLogDate(e.target.value)}
                  className="mt-1 block w-full border border-gray-30
                />
              </div>
```

```
            <div>
              <label className="block text-sm font-medium text-gra
              <input
                type="text"
                placeholder="e.g. Jane Doe"
                value={uploaderName}
                onChange={(e) => setUploaderName(e.target.value)}
                className="mt-1 block w-full border border-gray-30
              />
            </div>
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-
            <textarea
              rows="3"
              placeholder="Brief description of the file contents.
              value={description}
              onChange={(e) => setDescription(e.target.value)}
              className="mt-1 block w-full border border-gray-300
            />
          </div>

          <div>
            <button
              type="submit"
              disabled={isSubmitting}
              className={`w-full flex justify-center py-2 px-4 bor
            >
              {isSubmitting ? 'Uploading...' : 'Upload File'}
            </button>
          </div>
        </form>
      </div>

      {/* File Dashboard */}
      <div className="bg-white shadow sm:rounded-lg border border-
        <div className="px-4 py-5 border-b border-gray-200 sm:px-6
          <h3 className="text-lg leading-6 font-medium text-gray-9
          <button onClick={fetchFiles} className="text-sm text-blu
            Refresh List
          </button>
        </div>
        <div className="overflow-x-auto">
          <table className="min-w-full divide-y divide-gray-200">
            <thead className="bg-gray-50">
              <tr>
                <th scope="col" className="px-6 py-3 text-left tex
                <th scope="col" className="px-6 py-3 text-left tex
                <th scope="col" className="px-6 py-3 text-left tex
```

```
                <th scope="col" className="px-6 py-3 text-left te
                <th scope="col" className="px-6 py-3 text-right te
              </tr>
            </thead>
            <tbody className="bg-white divide-y divide-gray-200">
              {filesList.length === 0 ? (
                <tr>
                  <td colSpan="5" className="px-6 py-4 text-center
                    No files uploaded yet.
                  </td>
                </tr>
              ) : (
                filesList.map((f) => (
                  <tr key={f.id} className="hover:bg-gray-50">
                    <td className="px-6 py-4 whitespace-nowrap te
                      {f.originalName}
                    </td>
                    <td className="px-6 py-4 whitespace-nowrap te
                      {f.logDate || '-'}
                    </td>
                    <td className="px-6 py-4 whitespace-nowrap te
                      {f.uploaderName || '-'}
                    </td>
                    <td className="px-6 py-4 whitespace-nowrap te
                      {f.description || '-'}
                    </td>
                    <td className="px-6 py-4 whitespace-nowrap te
                      {/* Native anchor tag utilizing the backend'
                      <a
                        href={`${API_BASE}/download/${f.savedName}
                        download
                        className="inline-flex items-center px-3 p
                      >
                        Download
                      </a>
                    </td>
                  </tr>
                ))
              )}
            </tbody>
          </table>
        </div>
      </div>

    </div>
  </div>
);
}
```

Act as an expert UI/UX designer and React developer. I have a fully functional App.jsx for a File Upload Portal, but the UI is messy.
Please rewrite this component using Tailwind CSS and lucide-react icons to make it look like a highly professional, modern SaaS dashboard.
Design Requirements:

Layout: A clean, light-gray background (bg-gray-50). Center the content using a max-width container.
Header: A sleek top header or title area with a subtle icon.
The Upload Form: Style it as a white card with a subtle shadow (bg-white shadow-md rounded-lg). Make the input fields look modern (borders on focus, nice padding). Make the submit button a prominent, attractive color (e.g., Indigo or Blue) with a hover effect.
The Dashboard: Below the form, display the uploaded files not as a messy list, but as a clean, responsive Data Table. The table should have clear column headers (File Name, Date, Description, Uploader).
Download Button: Instead of an ugly HTML button, use a lucide-react Download icon styled beautifully.
Keep all of my existing fetch and state management logic exactly the same so nothing breaks!
Here is my current working code to upgrade:
"import React, { useState, useEffect } from 'react';

```
export default function App() {
  // --- State Hooks ---
  const [file, setFile] = useState(null);
  const [logDate, setLogDate] = useState('');
  const [description, setDescription] = useState('');
  const [uploaderName, setUploaderName] = useState('');

  const [filesList, setFilesList] = useState([]);
  const [status, setStatus] = useState({ type: '', message: '' });
  const [isSubmitting, setIsSubmitting] = useState(false);

  // --- API Base URL ---
  const API_BASE = 'http://localhost:3000';
```

```
  // --- Effects ---
  useEffect(() => {
    fetchFiles();
  }, []);

  // --- Functions ---
  const fetchFiles = async () => {
    try {
      const response = await fetch(`${API_BASE}/files`);
      if (!response.ok) throw new Error('Failed to fetch files');
      const data = await response.json();
      setFilesList(data);
    } catch (error) {
      console.error("Error fetching files:", error);
      setStatus({ type: 'error', message: 'Could not load the file dashboard.'
});
    }
  };

  const handleFileChange = (e) => {
    if (e.target.files && e.target.files.length > 0) {
      setFile(e.target.files[0]);
    }
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!file) {
      setStatus({ type: 'error', message: 'Please select a file to upload.' });
      return;
    }

    setIsSubmitting(true);
    setStatus({ type: '', message: '' });

    // Construct FormData for multipart/form-data payload
    const formData = new FormData();
```

```
    formData.append('file', file);
    formData.append('logDate', logDate);
    formData.append('description', description);
    formData.append('uploaderName', uploaderName);

    try {
      const response = await fetch(`${API_BASE}/upload`, {
        method: 'POST',
        body: formData,
      });

      const result = await response.json();

      if (!response.ok) {
        throw new Error(result.error || 'Upload failed');
      }

      setStatus({ type: 'success', message: 'File uploaded successfully!' });

      // Reset form fields
      setFile(null);
      setLogDate('');
      setDescription('');
      setUploaderName('');
      e.target.reset(); // Resets the file input visually

      // Refresh the dashboard
      fetchFiles();

    } catch (error) {
      console.error("Upload error:", error);
      setStatus({ type: 'error', message: error.message });
    } finally {
      setIsSubmitting(false);
    }
  };

  return (
```

```
    <div className="min-h-screen bg-gray-50 py-10 px-4 sm:px-6 lg:px-8">
      <div className="max-w-4xl mx-auto space-y-8">

        {/* Header */}
        <div className="text-center">
          <h1 className="text-3xl font-extrabold text-gray-900">File Upload Portal</h1>
          <p className="mt-2 text-sm text-gray-600">Upload and manage your system logs, PDFs, CSVs, and ZIP files safely.</p>
        </div>

        {/* Status Messages */}
        {status.message && (
          <div className={`p-4 rounded-md ${status.type === 'error' ? 'bg-red-50 text-red-700 border border-red-200' : 'bg-green-50 text-green-700 border border-green-200'}`}>
            <p className="text-sm font-medium">{status.message}</p>
          </div>
        )}

        {/* Upload Form */}
        <div className="bg-white py-8 px-6 shadow rounded-lg sm:px-10 border border-gray-200">
          <form onSubmit={handleSubmit} className="space-y-6">

            <div>
              <label className="block text-sm font-medium text-gray-700">Select File (.log, .pdf, .csv, .zip)</label>
              <div className="mt-1">
                <input
                  type="file"
                  accept=".log,.pdf,.csv,.zip"
                  onChange={handleFileChange}
                  className="block w-full text-sm text-gray-500 file:mr-4 file:py-2 file:px-4 file:rounded-md file:border-0 file:text-sm file:font-semibold file:bg-blue-50 file:text-blue-700 hover:file:bg-blue-100 border border-gray-300 rounded-md cursor-pointer"
```

```
            />
          </div>
        </div>

        <div className="grid grid-cols-1 gap-y-6 gap-x-4 sm:grid-cols-2">
          <div>
            <label className="block text-sm font-medium text-gray-700">Log Date</label>
            <input
              type="date"
              value={logDate}
              onChange={(e) => setLogDate(e.target.value)}
              className="mt-1 block w-full border border-gray-300 rounded-md shadow-sm py-2 px-3 focus:outline-none focus:ring-blue-500 focus:border-blue-500 sm:text-sm"
            />
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700">Uploader Name</label>
            <input
              type="text"
              placeholder="e.g. Jane Doe"
              value={uploaderName}
              onChange={(e) => setUploaderName(e.target.value)}
              className="mt-1 block w-full border border-gray-300 rounded-md shadow-sm py-2 px-3 focus:outline-none focus:ring-blue-500 focus:border-blue-500 sm:text-sm"
            />
          </div>
        </div>

        <div>
          <label className="block text-sm font-medium text-gray-700">Description</label>
          <textarea
```

```
            rows="3"
            placeholder="Brief description of the file contents..."
            value={description}
            onChange={(e) => setDescription(e.target.value)}
            className="mt-1 block w-full border border-gray-300 rounded-md shadow-sm py-2 px-3 focus:outline-none focus:ring-blue-500 focus:border-blue-500 sm:text-sm"
          />
        </div>

        <div>
          <button
            type="submit"
            disabled={isSubmitting}
            className={`w-full flex justify-center py-2 px-4 border border-transparent rounded-md shadow-sm text-sm font-medium text-white ${isSubmitting ? 'bg-blue-400 cursor-not-allowed' : 'bg-blue-600 hover:bg-blue-700'} focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-blue-500`}
          >
            {isSubmitting ? 'Uploading...' : 'Upload File'}
          </button>
        </div>
      </form>
    </div>

    {/* File Dashboard */}
    <div className="bg-white shadow sm:rounded-lg border border-gray-200">
      <div className="px-4 py-5 border-b border-gray-200 sm:px-6 flex justify-between items-center">
        <h3 className="text-lg leading-6 font-medium text-gray-900">File Dashboard</h3>
        <button onClick={fetchFiles} className="text-sm text-blue-600 hover:text-blue-500 font-medium">
          Refresh List
        </button>
      </div>
```

```
        <div className="overflow-x-auto">
          <table className="min-w-full divide-y divide-gray-200">
            <thead className="bg-gray-50">
              <tr>
                <th scope="col" className="px-6 py-3 text-left text-xs font-
medium text-gray-500 uppercase tracking-wider">File Name</th>
                <th scope="col" className="px-6 py-3 text-left text-xs font-
medium text-gray-500 uppercase tracking-wider">Log Date</th>
                <th scope="col" className="px-6 py-3 text-left text-xs font-
medium text-gray-500 uppercase tracking-wider">Uploader</th>
                <th scope="col" className="px-6 py-3 text-left text-xs font-
medium text-gray-500 uppercase tracking-wider">Description</th>
                <th scope="col" className="px-6 py-3 text-right text-xs font-
medium text-gray-500 uppercase tracking-wider">Action</th>
              </tr>
            </thead>
            <tbody className="bg-white divide-y divide-gray-200">
              {filesList.length === 0 ? (
                <tr>
                  <td colSpan="5" className="px-6 py-4 text-center text-sm
text-gray-500">
                    No files uploaded yet.
                  </td>
                </tr>
              ) : (
                filesList.map((f) => (
                  <tr key={f.id} className="hover:bg-gray-50">
                    <td className="px-6 py-4 whitespace-nowrap text-sm font-
medium text-gray-900 truncate max-w-[200px]" title={f.originalName}>
                      {f.originalName}
                    </td>
                    <td className="px-6 py-4 whitespace-nowrap text-sm text-
gray-500">
                      {f.logDate || '-'}
                    </td>
                    <td className="px-6 py-4 whitespace-nowrap text-sm text-
gray-500">
                      {f.uploaderName || '-'}
```

```
                </td>
                <td className="px-6 py-4 whitespace-nowrap text-sm text-gray-500 truncate max-w-[250px]" title={f.description}>
                  {f.description || '-'}
                </td>
                <td className="px-6 py-4 whitespace-nowrap text-right text-sm font-medium">
                  {/* Native anchor tag utilizing the backend's res.download() functionality */}
                  <a
                    href={`${API_BASE}/download/${f.savedName}`}
                    download
                    className="inline-flex items-center px-3 py-1.5 border border-transparent text-xs font-medium rounded-md text-blue-700 bg-blue-100 hover:bg-blue-200 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-blue-500"
                  >
                    Download
                  </a>
                </td>
              </tr>
            ))
          )}
        </tbody>
      </table>
    </div>
  </div>

  </div>
 </div>
 );
}
"
```

Here is the fully refactored `App.jsx`. I have kept all of your state management and `fetch` logic completely intact.

To achieve the modern SaaS aesthetic, I used Tailwind's `indigo` color palette for primary actions, added subtle shadows (`shadow-md`), rounded corners (`rounded-xl`), and improved the input focus states. I also integrated `lucide-react` for beautiful, consistent iconography (including the requested Download icon).

Prerequisites

Make sure you have `lucide-react` installed in your project:

Bash

```
npm install lucide-react
```

`App.jsx`

JavaScript

```
import React, { useState, useEffect } from 'react';
import {
  UploadCloud,
  Download,
  RefreshCw,
  AlertCircle,
  CheckCircle2,
  FileBox
} from 'lucide-react';

export default function App() {
  // --- State Hooks ---
  const [file, setFile] = useState(null);
  const [logDate, setLogDate] = useState('');
  const [description, setDescription] = useState('');
  const [uploaderName, setUploaderName] = useState('');

  const [filesList, setFilesList] = useState([]);
  const [status, setStatus] = useState({ type: '', message: '' });
  const [isSubmitting, setIsSubmitting] = useState(false);

  // --- API Base URL ---
  const API_BASE = 'http://localhost:3000';

  // --- Effects ---
  useEffect(() => {
    fetchFiles();
  }, []);
```

```
// --- Functions ---
const fetchFiles = async () => {
  try {
    const response = await fetch(`${API_BASE}/files`);
    if (!response.ok) throw new Error('Failed to fetch files');
    const data = await response.json();
    setFilesList(data);
  } catch (error) {
    console.error("Error fetching files:", error);
    setStatus({ type: 'error', message: 'Could not load the file c
  }
};

const handleFileChange = (e) => {
  if (e.target.files && e.target.files.length > 0) {
    setFile(e.target.files[0]);
  }
};

const handleSubmit = async (e) => {
  e.preventDefault();

  if (!file) {
    setStatus({ type: 'error', message: 'Please select a file to u
    return;
  }

  setIsSubmitting(true);
  setStatus({ type: '', message: '' });

  // Construct FormData for multipart/form-data payload
  const formData = new FormData();
  formData.append('file', file);
  formData.append('logDate', logDate);
  formData.append('description', description);
  formData.append('uploaderName', uploaderName);

  try {
    const response = await fetch(`${API_BASE}/upload`, {
      method: 'POST',
      body: formData,
    });

    const result = await response.json();

    if (!response.ok) {
      throw new Error(result.error || 'Upload failed');
    }
```

```
      setStatus({ type: 'success', message: 'File uploaded successfu

      // Reset form fields
      setFile(null);
      setLogDate('');
      setDescription('');
      setUploaderName('');
      e.target.reset(); // Resets the file input visually

      // Refresh the dashboard
      fetchFiles();

    } catch (error) {
      console.error("Upload error:", error);
      setStatus({ type: 'error', message: error.message });
    } finally {
      setIsSubmitting(false);
    }
  };

  return (
    <div className="min-h-screen bg-gray-50 py-12 px-4 sm:px-6 lg:p>
      <div className="max-w-5xl mx-auto space-y-10">

        {/* Header Area */}
        <div className="text-center flex flex-col items-center justi
          <div className="p-3 bg-indigo-100 rounded-full">
            <UploadCloud className="w-8 h-8 text-indigo-600" />
          </div>
          <h1 className="text-3xl font-extrabold text-gray-900 track
          <p className="text-base text-gray-500 max-w-xl">
            Securely upload, organize, and manage your system logs,
          </p>
        </div>

        {/* Status Messages */}
        {status.message && (
          <div className={`p-4 rounded-lg flex items-center shadow-s
            status.type === 'error' ? 'bg-red-50 text-red-800 borde
          }`}>
            {status.type === 'error' ? (
              <AlertCircle className="w-5 h-5 mr-3 flex-shrink-0" />
            ) : (
              <CheckCircle2 className="w-5 h-5 mr-3 flex-shrink-0" /
            )}
            <p className="text-sm font-medium">{status.message}</p>
          </div>
        )}

        {/* Upload Form Card */}
```

```
        <div className="bg-white shadow-md rounded-xl border border-
          <div className="px-6 py-5 border-b border-gray-100 bg-gray
            <h2 className="text-lg font-semibold text-gray-800 flex
              <FileBox className="w-5 h-5 mr-2 text-indigo-500" />
              New Upload
            </h2>
          </div>
          <div className="p-6 sm:p-8">
            <form onSubmit={handleSubmit} className="space-y-6">

              {/* File Input */}
              <div>
                <label className="block text-sm font-medium text-gra
                <div className="relative">
                  <input
                    type="file"
                    accept=".log,.pdf,.csv,.zip"
                    onChange={handleFileChange}
                    className="block w-full text-sm text-gray-500
                      file:mr-4 file:py-2.5 file:px-4
                      file:rounded-md file:border-0
                      file:text-sm file:font-medium
                      file:bg-indigo-50 file:text-indigo-700
                      hover:file:bg-indigo-100 hover:file:cursor-poi
                      border border-gray-300 rounded-lg cursor-point
                  />
                </div>
              </div>

              {/* Two Column Grid for Date and Name */}
              <div className="grid grid-cols-1 gap-y-6 gap-x-6 sm:gr
                <div>
                  <label className="block text-sm font-medium text-g
                  <input
                    type="date"
                    value={logDate}
                    onChange={(e) => setLogDate(e.target.value)}
                    className="block w-full border border-gray-300 r
                  />
                </div>

                <div>
                  <label className="block text-sm font-medium text-g
                  <input
                    type="text"
                    placeholder="e.g. Jane Doe"
                    value={uploaderName}
                    onChange={(e) => setUploaderName(e.target.value)
                    className="block w-full border border-gray-300 r
                  />
```

```
            </div>
          </div>

          {/* Description */}
          <div>
            <label className="block text-sm font-medium text-gra
            <textarea
              rows="3"
              placeholder="Brief description of the file content
              value={description}
              onChange={(e) => setDescription(e.target.value)}
              className="block w-full border border-gray-300 rou
            />
          </div>

          {/* Submit Button */}
          <div className="pt-2">
            <button
              type="submit"
              disabled={isSubmitting}
              className={`w-full flex justify-center items-cente
                isSubmitting
                  ? 'bg-indigo-400 cursor-not-allowed'
                  : 'bg-indigo-600 hover:bg-indigo-700 hover:sha
              }`}
            >
              {isSubmitting ? (
                <>
                  <RefreshCw className="w-4 h-4 mr-2 animate-spi
                  Uploading...
                </>
              ) : (
                <>
                  <UploadCloud className="w-4 h-4 mr-2" />
                  Upload File
                </>
              )}
            </button>
          </div>
        </form>
      </div>
    </div>

    {/* File Dashboard */}
    <div className="bg-white shadow-md rounded-xl border border-
      {/* Dashboard Header */}
      <div className="px-6 py-5 border-b border-gray-100 flex ju
        <h3 className="text-lg font-semibold text-gray-800">File
        <button
          onClick={fetchFiles}
```

```
            className="flex items-center text-sm text-indigo-600 h
          >
            <RefreshCw className="w-4 h-4 mr-1.5" />
            Refresh
          </button>
        </div>

        {/* Data Table */}
        <div className="overflow-x-auto">
          <table className="min-w-full divide-y divide-gray-200">
            <thead className="bg-white">
              <tr>
                <th scope="col" className="px-6 py-4 text-left te
                <th scope="col" className="px-6 py-4 text-left te
                <th scope="col" className="px-6 py-4 text-left te
                <th scope="col" className="px-6 py-4 text-left te
                <th scope="col" className="px-6 py-4 text-center t
              </tr>
            </thead>
            <tbody className="bg-white divide-y divide-gray-100">
              {filesList.length === 0 ? (
                <tr>
                  <td colSpan="5" className="px-6 py-10 text-cente
                    <div className="flex flex-col items-center jus
                      <FileBox className="w-10 h-10 text-gray-300
                      <p>No files have been uploaded yet.</p>
                    </div>
                  </td>
                </tr>
              ) : (
                filesList.map((f) => (
                  <tr key={f.id} className="hover:bg-gray-50 trans
                    <td className="px-6 py-4 whitespace-nowrap te
                      {f.originalName}
                    </td>
                    <td className="px-6 py-4 whitespace-nowrap te
                      {f.logDate ? new Date(f.logDate).toLocaleDat
                    </td>
                    <td className="px-6 py-4 whitespace-nowrap te
                      {f.uploaderName || '-'}
                    </td>
                    <td className="px-6 py-4 whitespace-nowrap te
                      {f.description || '-'}
                    </td>
                    <td className="px-6 py-4 whitespace-nowrap te
                      <a
                        href={`${API_BASE}/download/${f.savedName}
                        download
                        title="Download file"
                        className="inline-flex items-center justif
```

```
                        >
                          <Download className="w-5 h-5" />
                        </a>
                      </td>
                    </tr>
                  ))
                )}
              </tbody>
            </table>
          </div>
        </div>

      </div>
    </div>
```

Act as an expert Python developer. I need to upgrade my existing Python CSV Plotter CLI script with two advanced features: URL ingestion and automated data cleaning.
Please rewrite the script using pandas and matplotlib to follow this exact flow:

Data Source Selection: When the script starts, ask the user in the terminal: "Would you like to (1) Select a local file, or (2) Enter a URL for a CSV?". If they choose 1, open the tkinter.filedialog to let them pick the file. If they choose 2, prompt them to paste the URL. Load the selected source into a pandas DataFrame.
Automated Data Cleaning: Once loaded, check the DataFrame for any missing values (NaN). If missing values exist, print a warning to the terminal showing the total number of missing cells. Then, prompt the user: "Missing data detected. Do you want to (1) Drop rows with missing data, or (2) Fill missing data with 0?". Apply the chosen cleaning method.
Column Selection: Print a numbered list of all available columns. Prompt the user to select the X-axis and Y-axis by typing their corresponding numbers.
Graph Generation: Ask if they want a Line chart or a Bar graph, and display it using matplotlib.
Robust Error Handling: Include try-except blocks. For example, handle what happens if the user provides a dead URL, or if they select a column with text data for the Y-axis.
Keep the code modular, well-commented, and entirely within a single Python file so it remains highly portable.

Here is the upgraded version of your CSV plotter utility. I have modularized the new features—URL ingestion and automated data cleaning—into their own functions to keep the code clean and maintainable.

`pandas` natively supports reading CSVs directly from URLs, which makes the implementation straightforward, but I have wrapped it in robust `try-except` blocks to handle network issues or 404 errors cleanly.

Prerequisites

Ensure your environment has the necessary libraries installed. (`tkinter` is included with standard Python installations, so no need to pip install it).

Bash

```
pip install pandas matplotlib
```

The Upgraded Python Script

Save this as `advanced_csv_plotter.py` and run it from your terminal.

Python

```
import sys
import pandas as pd
import matplotlib.pyplot as plt
import tkinter as tk
from tkinter import filedialog
import urllib.error

def select_local_file() -> str:
    """Opens a native file dialog for the user to select a CSV file.
    root = tk.Tk()
    root.withdraw() # Hide the main window
    root.attributes('-topmost', True) # Force on top

    print("Opening file picker...")
    file_path = filedialog.askopenfilename(
        title="Select a CSV File",
        filetypes=(("CSV files", "*.csv"), ("All files", "*.*"))
    )
    return file_path

def get_data_source() -> str:
    """Prompts the user to choose between a local file or a URL."""
    while True:
        print("\nData Source Selection:")
        print("1. Select a local file")
        print("2. Enter a URL for a CSV")
        choice = input("Enter your choice (1 or 2): ").strip()

        if choice == '1':
            source = select_local_file()
            if not source:
                print("No file selected. Exiting.")
                sys.exit()
            return source
```

```
        elif choice == '2':
            source = input("Paste the CSV URL: ").strip()
            if not source:
                print("No URL provided. Exiting.")
                sys.exit()
            return source
        else:
            print("Invalid choice. Please enter 1 or 2.")

def load_data(source: str) -> pd.DataFrame:
    """Loads the CSV data from a file path or URL into a DataFrame."
    print(f"\nAttempting to load data from: {source}")
    try:
        df = pd.read_csv(source)
        return df
    except urllib.error.URLError as e:
        print(f"\n[Error] Network issue or invalid URL: {e.reason}")
        sys.exit()
    except urllib.error.HTTPError as e:
        print(f"\n[Error] HTTP Error encountered: {e.code} - {e.reas
        sys.exit()
    except FileNotFoundError:
        print("\n[Error] The specified local file was not found.")
        sys.exit()
    except pd.errors.EmptyDataError:
        print("\n[Error] The selected CSV file is empty.")
        sys.exit()
    except Exception as e:
        print(f"\n[Error] Failed to read the CSV data: {e}")
        sys.exit()

def clean_data(df: pd.DataFrame) -> pd.DataFrame:
    """Checks for NaN values and prompts the user to resolve them if
    missing_cells = df.isnull().sum().sum()

    if missing_cells == 0:
        print("Data check passed: No missing values detected.")
        return df

    print(f"\n[Warning] Missing data detected: {missing_cells} empty

    while True:
        print("How would you like to handle the missing data?")
        print("1. Drop rows with missing data")
        print("2. Fill missing data with 0")
        choice = input("Enter your choice (1 or 2): ").strip()

        if choice == '1':
            initial_rows = len(df)
            df = df.dropna()
```

```
            dropped_count = initial_rows - len(df)
            print(f"Dropped {dropped_count} row(s) containing missir
            return df
        elif choice == '2':
            df = df.fillna(0)
            print("Filled all missing cells with 0.")
            return df
        else:
            print("Invalid choice. Please enter 1 or 2.")

def get_column_choice(columns: list, axis_name: str) -> str:
    """Prompts the user to select a column by its numbered index."""
    while True:
        try:
            choice = int(input(f"Select the column number for the {a
            if 1 <= choice <= len(columns):
                return columns[choice - 1]
            else:
                print(f"Invalid choice. Please enter a number betwee
        except ValueError:
            print("Invalid input. Please enter a valid number.")

def get_plot_type() -> str:
    """Prompts the user to choose between a line or bar graph."""
    while True:
        choice = input("Choose plot type ('line' or 'bar'): ").strip
        if choice in ['line', 'bar']:
            return choice
        print("Invalid choice. Please type 'line' or 'bar'.")

def plot_data(df: pd.DataFrame, x_col: str, y_col: str, plot_type: s
    """Generates and displays the requested matplotlib plot."""
    # Robust error handling for non-numeric Y-axis data
    if not pd.api.types.is_numeric_dtype(df[y_col]):
        print(f"\n[Warning] The selected Y-axis column '{y_col}' cor
        print("Matplotlib may fail, plot categorically, or produce a
        print("Attempting to plot anyway...\n")

    plt.figure(figsize=(10, 6))

    try:
        if plot_type == 'line':
            plt.plot(df[x_col], df[y_col], marker='o', linestyle='-'
        elif plot_type == 'bar':
            plt.bar(df[x_col], df[y_col], color='indigo')

        plt.title(f"{plot_type.capitalize()} Plot: {y_col} vs {x_col
        plt.xlabel(x_col)
        plt.ylabel(y_col)
```

```
        # Rotate X-axis labels to prevent overlap
        plt.xticks(rotation=45, ha='right')
        plt.tight_layout()

        plt.show()

    except Exception as e:
        print(f"\n[Error] A fatal error occurred while generating th

def main():
    print("--- Advanced CSV Data Plotting Utility ---")

    # 1. Select data source and load data
    source = get_data_source()
    raw_df = load_data(source)

    if raw_df.empty:
        print("[Error] The loaded CSV data is completely empty. Exit
        sys.exit()

    # 2. Automated Data Cleaning
    df = clean_data(raw_df)

    # 3. Print available columns
    print("\nAvailable columns:")
    columns = df.columns.tolist()
    for idx, col in enumerate(columns, start=1):
        print(f"  {idx}. {col}")
    print("-" * 40)

    # 4. Get user selections
    x_col = get_column_choice(columns, "X")
    y_col = get_column_choice(columns, "Y")
    print("-" * 40)
    plot_type = get_plot_type()

    # 5. Plot the data
    print("\nGenerating plot... (Close the plot window to exit the s
    plot_data(df, x_col, y_col, plot_type)

if __name__ == "__main__":
```

Act as an expert Full-Stack Developer. I need to upgrade my React/Node File Upload Portal with several advanced features: a Drag-and-Drop zone, Toast notifications, file size limits, and a Document Preview Modal.

1. Backend (server.js) Upgrades:

Keep the existing multer setup, but add a file size limit of 5MB. If a file exceeds this, return an HTTP 400 error with a clean JSON message.
Add a new endpoint: GET /preview/:filename. Unlike the download endpoint, this should use res.sendFile() to serve the file inline so the browser can read it for the preview feature.
2. Frontend (App.jsx) Upgrades:

Drag-and-Drop: Replace the standard file input with a beautiful Tailwind-styled Drag-and-Drop zone. It should highlight when a file is dragged over it. Use standard React onDragOver and onDrop events.
Toast Notifications: Integrate react-hot-toast. Replace all native browser alert() calls with elegant success/error toasts.
Preview Modal: In the File Dashboard, add a "Preview" button (using a lucide-react Eye icon) next to the Download button. When clicked, it should open a centered modal overlay.
If the file is a .pdf, render it inside an <iframe src="http://localhost:3000/preview/FILENAME">.
If the file is .csv or .log, use fetch() to grab the text from the preview endpoint and display it in a scrollable <code> or <pre> block.
If the file is .zip, display a stylized message saying "Preview not available for ZIP archives."
Please provide the complete updated server.js and App.jsx code, and remind me to npm install react-hot-toast. Ensure all Tailwind classes remain intact so the UI stays looking like a premium SaaS dashboard.

Here is the complete, upgraded code for both your backend and frontend.

Prerequisites

Before running the upgraded frontend, you need to install the new toast notification library. Run this in your React project directory:

Bash

```
npm install react-hot-toast
```

---

1. The Backend (`server.js`)

I have added the 5MB limit to the Multer configuration and created the new `/preview/:filename` endpoint using `res.sendFile()` so the browser can interpret the file inline.

JavaScript

```
const express = require('express');
const multer = require('multer');
const cors = require('cors');
const fs = require('fs');
const path = require('path');

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Paths
const UPLOADS_DIR = path.join(__dirname, 'uploads');
const METADATA_FILE = path.join(__dirname, 'metadata.json');

// --- Initialization ---
if (!fs.existsSync(UPLOADS_DIR)) fs.mkdirSync(UPLOADS_DIR, { recursi
if (!fs.existsSync(METADATA_FILE)) fs.writeFileSync(METADATA_FILE, 

const readMetadata = () => {
    try {
        return JSON.parse(fs.readFileSync(METADATA_FILE, 'utf8'));
    } catch (err) {
        return [];
    }
};

const writeMetadata = (data) => {
    try {
        fs.writeFileSync(METADATA_FILE, JSON.stringify(data, null, 2
    } catch (err) {}
};

// --- Multer Configuration ---
const storage = multer.diskStorage({
    destination: (req, file, cb) => cb(null, UPLOADS_DIR),
    filename: (req, file, cb) => {
        const uniqueSuffix = Date.now() + '-' + Math.round(Math.rand
        const ext = path.extname(file.originalname);
        const baseName = path.basename(file.originalname, ext);
```

```
        cb(null, `${baseName}-${uniqueSuffix}${ext}`);
    }
});

const fileFilter = (req, file, cb) => {
    const allowedExtensions = ['.log', '.pdf', '.csv', '.zip'];
    const ext = path.extname(file.originalname).toLowerCase();

    if (allowedExtensions.includes(ext)) {
        cb(null, true);
    } else {
        cb(new Error('Invalid file type. Only .log, .pdf, .csv, and
    }
};

const upload = multer({
    storage: storage,
    fileFilter: fileFilter,
    limits: { fileSize: 5 * 1024 * 1024 } // 5MB Limit
});

// --- API Endpoints ---

// POST /upload
app.post('/upload', (req, res) => {
    const uploadHandler = upload.single('file');

    uploadHandler(req, res, (err) => {
        if (err instanceof multer.MulterError) {
            if (err.code === 'LIMIT_FILE_SIZE') {
                return res.status(400).json({ error: 'File exceeds t
            }
            return res.status(400).json({ error: err.message });
        } else if (err) {
            return res.status(400).json({ error: err.message });
        }

        if (!req.file) return res.status(400).json({ error: 'No file

        const { logDate, description, uploaderName } = req.body;
        const newEntry = {
            id: Date.now().toString(),
            originalName: req.file.originalname,
            savedName: req.file.filename,
            size: req.file.size,
            logDate: logDate || '',
            description: description || '',
            uploaderName: uploaderName || '',
            uploadTimestamp: new Date().toISOString()
        };
```

```
        const metadata = readMetadata();
        metadata.push(newEntry);
        writeMetadata(metadata);

        res.status(201).json({ message: 'File uploaded successfully!
    });
});

// GET /files
app.get('/files', (req, res) => res.json(readMetadata()));

// GET /preview/:filename (Inline viewing)
app.get('/preview/:filename', (req, res) => {
    const filePath = path.join(UPLOADS_DIR, req.params.filename);
    if (!filePath.startsWith(UPLOADS_DIR)) return res.status(403).js

    if (fs.existsSync(filePath)) {
        res.sendFile(filePath);
    } else {
        res.status(404).json({ error: 'File not found.' });
    }
});

// GET /download/:filename (Forced download)
app.get('/download/:filename', (req, res) => {
    const filePath = path.join(UPLOADS_DIR, req.params.filename);
    if (!filePath.startsWith(UPLOADS_DIR)) return res.status(403).js

    if (fs.existsSync(filePath)) {
        res.download(filePath);
    } else {
        res.status(404).json({ error: 'File not found.' });
    }
});

app.listen(PORT, () => {
    console.log(`Server is running on http://localhost:${PORT}`);
});
```

---

2. The Frontend (`App.jsx`)

I have integrated `react-hot-toast`, built out the Drag-and-Drop functionality, and added the Preview Modal logic.

JavaScript

```
import React, { useState, useEffect, useRef } from 'react';
import {
  UploadCloud, Download, RefreshCw, FileBox, Eye, X, FileText, Check
} from 'lucide-react';
import toast, { Toaster } from 'react-hot-toast';

export default function App() {
  const [file, setFile] = useState(null);
  const [logDate, setLogDate] = useState('');
  const [description, setDescription] = useState('');
  const [uploaderName, setUploaderName] = useState('');
  const [filesList, setFilesList] = useState([]);
  const [isSubmitting, setIsSubmitting] = useState(false);

  // Drag and Drop State
  const [isDragActive, setIsDragActive] = useState(false);
  const fileInputRef = useRef(null);

  // Preview Modal State
  const [previewFile, setPreviewFile] = useState(null);
  const [previewContent, setPreviewContent] = useState('');
  const [isPreviewLoading, setIsPreviewLoading] = useState(false);

  const API_BASE = 'http://localhost:3000';

  useEffect(() => {
    fetchFiles();
  }, []);

  const fetchFiles = async () => {
    try {
      const response = await fetch(`${API_BASE}/files`);
      if (!response.ok) throw new Error('Failed to fetch files');
      setFilesList(await response.json());
    } catch (error) {
      toast.error('Could not load the file dashboard.');
    }
  };

  // --- Drag and Drop Handlers ---
  const handleDragOver = (e) => {
    e.preventDefault();
    setIsDragActive(true);
  };

  const handleDragLeave = (e) => {
    e.preventDefault();
    setIsDragActive(false);
  };
```

```
  const handleDrop = (e) => {
    e.preventDefault();
    setIsDragActive(false);
    if (e.dataTransfer.files && e.dataTransfer.files.length > 0) {
      validateAndSetFile(e.dataTransfer.files[0]);
    }
  };

  const handleFileChange = (e) => {
    if (e.target.files && e.target.files.length > 0) {
      validateAndSetFile(e.target.files[0]);
    }
  };

  const validateAndSetFile = (selectedFile) => {
    const validExtensions = ['.log', '.pdf', '.csv', '.zip'];
    const extension = selectedFile.name.substring(selectedFile.name.

    if (!validExtensions.includes(extension)) {
      toast.error('Invalid file type. Only .log, .pdf, .csv, and .zi
      return;
    }
    if (selectedFile.size > 5 * 1024 * 1024) {
      toast.error('File exceeds the 5MB limit.');
      return;
    }
    setFile(selectedFile);
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!file) return toast.error('Please select a file to upload.')

    setIsSubmitting(true);
    const formData = new FormData();
    formData.append('file', file);
    formData.append('logDate', logDate);
    formData.append('description', description);
    formData.append('uploaderName', uploaderName);

    try {
      const response = await fetch(`${API_BASE}/upload`, { method: '
      const result = await response.json();

      if (!response.ok) throw new Error(result.error || 'Upload fail

      toast.success('File uploaded successfully!');
      setFile(null);
      setLogDate('');
      setDescription('');
```

```
      setUploaderName('');
      if (fileInputRef.current) fileInputRef.current.value = '';
      fetchFiles();
    } catch (error) {
      toast.error(error.message);
    } finally {
      setIsSubmitting(false);
    }
  };

  // --- Preview Handlers ---
  const handlePreview = async (f) => {
    setPreviewFile(f);
    const ext = f.originalName.substring(f.originalName.lastIndexOf(

    if (ext === '.csv' || ext === '.log') {
      setIsPreviewLoading(true);
      try {
        const res = await fetch(`${API_BASE}/preview/${f.savedName}`
        if (!res.ok) throw new Error('Failed to load preview');
        setPreviewContent(await res.text());
      } catch (err) {
        setPreviewContent('Error loading file content.');
      } finally {
        setIsPreviewLoading(false);
      }
    }
  };

  const closePreview = () => {
    setPreviewFile(null);
    setPreviewContent('');
  };

  return (
    <div className="min-h-screen bg-gray-50 py-12 px-4 sm:px-6 lg:px
      <Toaster position="top-right" />

      <div className="max-w-5xl mx-auto space-y-10">

        {/* Header */}
        <div className="text-center flex flex-col items-center justi
          <div className="p-3 bg-indigo-100 rounded-full">
            <UploadCloud className="w-8 h-8 text-indigo-600" />
          </div>
          <h1 className="text-3xl font-extrabold text-gray-900 track
          <p className="text-base text-gray-500 max-w-xl">Securely u
        </div>

        {/* Upload Form */}
```

```
        <div className="bg-white shadow-md rounded-xl border border-
          <div className="px-6 py-5 border-b border-gray-100 bg-gra
            <h2 className="text-lg font-semibold text-gray-800 flex
              <FileBox className="w-5 h-5 mr-2 text-indigo-500" />
              New Upload
            </h2>
          </div>
          <div className="p-6 sm:p-8">
            <form onSubmit={handleSubmit} className="space-y-6">

              {/* Drag and Drop Zone */}
              <div>
                <label className="block text-sm font-medium text-gra
                <div
                  onDragOver={handleDragOver}
                  onDragLeave={handleDragLeave}
                  onDrop={handleDrop}
                  onClick={() => fileInputRef.current?.click()}
                  className={`mt-1 flex justify-center px-6 pt-5 pb-
                    isDragActive ? 'border-indigo-500 bg-indigo-50'
                  }`}
                >
                  <div className="space-y-1 text-center">
                    {file ? (
                      <div className="flex flex-col items-center jus
                        <CheckCircle className="mx-auto h-10 w-10 mb
                        <span className="text-sm font-medium text-gr
                        <span className="text-xs text-gray-500">{(fi
                      </div>
                    ) : (
                      <>
                        <UploadCloud className={`mx-auto h-12 w-12 $
                        <div className="flex text-sm text-gray-600 j
                          <span className="relative font-medium text
                            Upload a file
                          </span>
                          <p className="pl-1">or drag and drop</p>
                        </div>
                        <p className="text-xs text-gray-500">.LOG, .
                      </>
                    )}
                    <input
                      ref={fileInputRef}
                      type="file"
                      accept=".log,.pdf,.csv,.zip"
                      onChange={handleFileChange}
                      className="hidden"
                    />
                  </div>
                </div>
```

```
            </div>

            {/* Text Fields */}
            <div className="grid grid-cols-1 gap-y-6 gap-x-6 sm:gr
              <div>
                <label className="block text-sm font-medium text-g
                <input type="date" value={logDate} onChange={(e) =
              </div>
              <div>
                <label className="block text-sm font-medium text-g
                <input type="text" placeholder="e.g. Jane Doe" val
              </div>
            </div>

            <div>
              <label className="block text-sm font-medium text-gra
              <textarea rows="3" placeholder="Brief description...
            </div>

            <button type="submit" disabled={isSubmitting} classNam
              {isSubmitting ? <><RefreshCw className="w-4 h-4 mr-2
            </button>
          </form>
        </div>
      </div>

      {/* File Dashboard */}
      <div className="bg-white shadow-md rounded-xl border border-
        <div className="px-6 py-5 border-b border-gray-100 flex ju
          <h3 className="text-lg font-semibold text-gray-800">File
          <button onClick={fetchFiles} className="flex items-cente
            <RefreshCw className="w-4 h-4 mr-1.5" /> Refresh
          </button>
        </div>

        <div className="overflow-x-auto">
          <table className="min-w-full divide-y divide-gray-200">
            <thead className="bg-white">
              <tr>
                <th className="px-6 py-4 text-left text-xs font-se
                <th className="px-6 py-4 text-left text-xs font-se
                <th className="px-6 py-4 text-left text-xs font-se
                <th className="px-6 py-4 text-center text-xs font-
              </tr>
            </thead>
            <tbody className="bg-white divide-y divide-gray-100">
              {filesList.length === 0 ? (
                <tr><td colSpan="4" className="px-6 py-10 text-cen
              ) : (
                filesList.map((f) => (
```

```
                <tr key={f.id} className="hover:bg-gray-50 trans
                  <td className="px-6 py-4 whitespace-nowrap te
                  <td className="px-6 py-4 whitespace-nowrap te
                  <td className="px-6 py-4 whitespace-nowrap te
                  <td className="px-6 py-4 whitespace-nowrap te
                    <button onClick={() => handlePreview(f)} tit
                      <Eye className="w-5 h-5" />
                    </button>
                    <a href={`${API_BASE}/download/${f.savedName
                      <Download className="w-5 h-5" />
                    </a>
                  </td>
                </tr>
              ))
            )}
          </tbody>
        </table>
      </div>
    </div>
  </div>

  {/* Preview Modal */}
  {previewFile && (
    <div className="fixed inset-0 z-50 flex items-center justify
      <div className="bg-white rounded-xl shadow-2xl w-full max-
        <div className="px-6 py-4 border-b border-gray-100 flex
          <h3 className="text-lg font-semibold text-gray-800 fle
            <FileText className="w-5 h-5 mr-2 text-indigo-500 fl
            {previewFile.originalName}
          </h3>
          <button onClick={closePreview} className="text-gray-40
            <X className="w-6 h-6" />
          </button>
        </div>

        <div className="flex-1 overflow-auto bg-gray-50 p-6">
          {previewFile.originalName.toLowerCase().endsWith('.pdf
            <iframe
              src={`${API_BASE}/preview/${previewFile.savedName}
              className="w-full h-[60vh] rounded-lg border borde
              title="PDF Preview"
            />
          ) : previewFile.originalName.toLowerCase().endsWith('.
            <div className="flex flex-col items-center justify-c
              <FileBox className="w-16 h-16 text-gray-300 mb-4" 
              <p className="text-lg font-medium">Preview not ava
              <p className="text-sm">Please download the file to
            </div>
          ) : (
            <div className="bg-gray-900 rounded-lg p-4 shadow-in
```

```
                {isPreviewLoading ? (
                  <div className="flex items-center justify-center
                    <RefreshCw className="w-6 h-6 animate-spin mr-
                  </div>
                ) : (
                  <pre className="text-sm text-green-400 font-mono
                    <code>{previewContent}</code>
                  </pre>
                )}
              </div>
            )}
          </div>
        </div>
      </div>
```

Steps to wrap react applications in a docker.

Wrapping a React application in Docker is an excellent way to ensure it runs consistently across any environment. The industry best practice for doing this is using a **multi-stage build**.

In a multi-stage build, the first stage uses Node.js to install dependencies and build the static HTML/CSS/JS files. The second stage uses a lightweight web server (like Nginx) to serve those static files. This keeps your final Docker image incredibly small and secure, as it won't include Node.js or your raw source code.

Here is the step-by-step guide to containerizing your React app.

Step 1: Create a `.dockerignore` file

Just like `.gitignore`, this file tells Docker which files and folders to ignore when copying your project into the container. This drastically speeds up the build process.

Create a file named `.dockerignore` in the root of your project and add the following:

Plaintext

```
node_modules
build
dist
```

```
.git
.env
```

Step 2: Create the `Dockerfile`

Create a file named `Dockerfile` (no extension) in the root of your project. This is the blueprint for your container.

Dockerfile

```
# ========================================
# Stage 1: Build the React application
# ========================================
FROM node:18-alpine as build

# Set the working directory inside the container
WORKDIR /app

# Copy package.json and package-lock.json first
# This takes advantage of Docker's layer caching for faster builds
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy the rest of your app's source code
COPY . .

# Build the React app
# (Note: If using Vite, this output folder is usually 'dist'. For Cr
RUN npm run build

# ========================================
# Stage 2: Serve the application with Nginx
# ========================================
FROM nginx:alpine

# Copy the build output from Stage 1 to Nginx's default public folde
# Change /app/build to /app/dist if you are using Vite!
COPY --from=build /app/build /usr/share/nginx/html

# Expose port 80 to the outside world
EXPOSE 80

# Start Nginx server
CMD ["nginx", "-g", "daemon off;"]
```

Step 3: Build the Docker Image

Open your terminal, navigate to your project's root directory (where the `Dockerfile` is), and run the build command. The `-t` flag tags your image with a readable name.

Bash

```
docker build -t my-react-app .
```

(Don't forget the period `.` at the end—it tells Docker to look in the current directory).

Step 4: Run the Docker Container

Once the image is built, you can spin up a container from it.

Bash

```
docker run -d -p 8080:80 --name react-container my-react-app
```

- `-d`: Runs the container in "detached" mode (in the background).
- `-p 8080:80`: Maps port 8080 on your local machine to port 80 inside the Nginx container.
- `--name`: Gives your running container a friendly name.

You can now open your browser and go to **http://localhost:8080** to see your React app running in Docker!

Optional: Use Docker Compose (Highly Recommended)

If you don't want to type out those long terminal commands every time, or if your React app eventually needs to talk to a backend API or database, `docker-compose` is the way to go.

Create a `docker-compose.yml` file in your root directory:

YAML

```
version: '3.8'
```

```
services:
  frontend:
    build: .
    ports:
      - "8080:80"
    container_name: react-frontend
    restart: unless-stopped
```

Now, instead of the commands in Steps 3 and 4, you simply run:

Bash

```
docker-compose up -d --build
```

A quick note on Environment Variables

Because React applications are built into static files, standard environment variables (like `.env`) are "baked in" during the `npm run build` step (Stage 1). If you need dynamic environment variables that change *after* the container is built (e.g., pointing to a Dev API vs a Prod API at runtime), you will need to implement a runtime configuration script or inject them via a window object in your `index.html`.